

UNIVERSITÀ DEGLI STUDI DI BARI ALDO MORO | DIPARTIMENTO DI INFORMATICA



Documentazione Caso di Studio Ingegneria della Conoscenza A.A. 2025/26 Gestione Intelligente delle Emergenze Urbane

Gruppo di lavoro

- Vincenzo Campanelli, 758276, v.campanelli5@studenti.uniba.it

Repository GitHub

- https://github.com/Atlas2609/emergenze

Sommario

Introduzione 3
Base di conoscenza (Prolog) 4
Sommario 4
Strumenti utilizzati 4
Decisioni di progetto 4
Valutazione 5
Ontologia (OWL) 5
Sommario 5
Strumenti utilizzati 5
Decisioni di progetto 5
Valutazione 6
Ricerca del percorso (A*) 6
Sommario 6
Strumenti utilizzati 6
Decisioni di progetto 6
Valutazione 7
Allocazione dei mezzi (CSP) 7
Sommario 7
Strumenti utilizzati 7
Decisioni di progetto 7
Valutazione 8
Apprendimento supervisionato 9
Sommario 9
Strumenti utilizzati 9
Decisioni di progetto 9
Valutazione 10
Ragionamento con incertezza (rete bayesiana) 12
Sommario 12
Strumenti utilizzati 12
Decisioni di progetto 12
Valutazione 12
Conclusioni 13
Riferimenti bibliografici 14

Introduzione

Una centrale operativa di soccorso deve gestire più emergenze contemporanee in una città: per ciascuna deve decidere quale mezzo inviare (ambulanza, autopompa o autoscala), quale percorso fargli seguire e se è probabile che servano rinforzi. Le condizioni cambiano nel tempo: pioggia forte o alluvione allagano le strade, che diventano impraticabili per i mezzi leggeri, e nebbia e pioggia rallentano tutti i veicoli.

Il sistema sviluppato affronta il problema combinando diverse tecniche del corso, ognuna dove è più adatta:

- **Base di conoscenza in Prolog**: descrive la mappa, i mezzi e le regole di transitabilità; le condizioni meteo vengono aggiornate a runtime con `assert`/`retract`.
- **Ontologia OWL**: formalizza il vocabolario del dominio; il ragionatore HermiT classifica i mezzi e se ne verifica la coerenza con la KB.
- **Ricerca A***: calcola il percorso di costo minimo interrogando la KB a ogni espansione.
- **Problema di soddisfacimento di vincoli**: alloca i mezzi alle emergenze; l'ottimo esatto (AC-3, backtracking, branch and bound) è confrontato con la ricerca locale (Random Restart Hill Climbing e Simulated Annealing).
- **Apprendimento supervisionato e reti bayesiane**: stimano la probabilità che un'emergenza richieda supporto aggiuntivo, valutati con nested cross-validation.

Il flusso di una simulazione (`main.py`) è il seguente: si carica la KB, si costruisce e si classifica l'ontologia, si genera uno scenario (emergenze, mezzi, meteo, fascia oraria) che viene asserito nella KB, A* calcola la matrice dei costi mezzo → emergenza, il CSP sceglie l'allocazione e infine il modello appreso stima, per ogni emergenza, la probabilità di dover inviare rinforzi. Le feature usate per la predizione vengono dalla KB (meteo del nodo, zona, fascia oraria) e da A* (ritardo di arrivo del mezzo assegnato): i moduli non lavorano isolati ma si passano i risultati.

Architettura del sistema

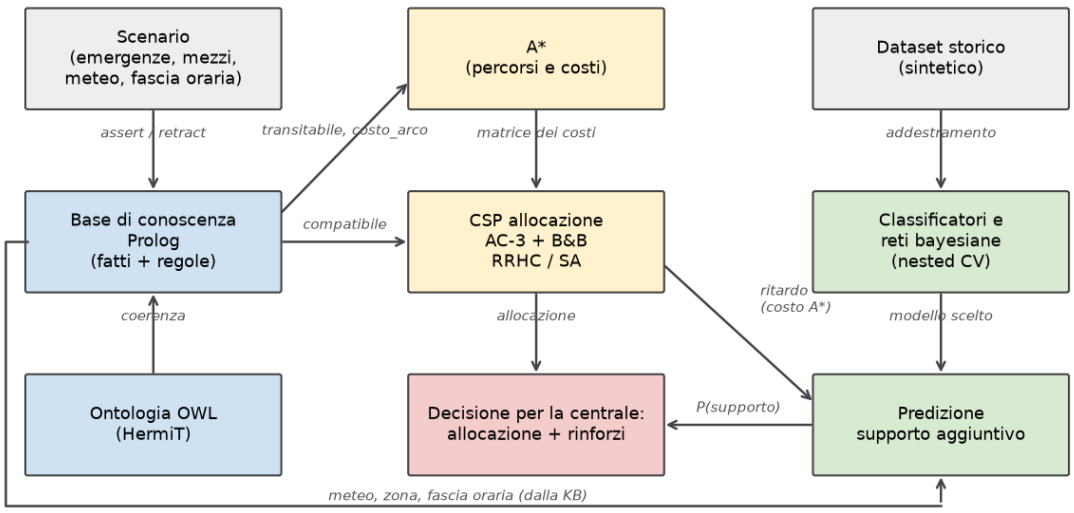


Figura 1 – Architettura del sistema e flusso dei dati tra i moduli.

Tutte le valutazioni riportate in questo documento si riproducono con `python esperimenti.py` e i diagrammi con `python figure_relazione.py`; il codice è coperto da 65 test unitari (`python -m pytest test`).

Base di conoscenza (Prolog)

Sommario

La KB (`prolog/mappa_emergenze.pl`) rappresenta una città di 15 nodi e 26 archi divisa in 4 zone. Contiene fatti statici (nodi, coordinate, archi con distanza, zone, tipi di mezzo con categoria e capacità, tipi di emergenza, compatibilità mezzo/emergenza, basi operative) e fatti dinamici (meteo di ogni zona, fascia oraria corrente). Le regole derivano la conoscenza che non viene memorizzata: allagamento dei nodi, transitabilità di un arco per un veicolo e costo effettivo dell'arco.

Strumenti utilizzati

SWI-Prolog 10 e la libreria pyswip, che consente di interrogare la KB da Python (`logica/kb_interface.py`).

Decisioni di progetto

Le regole principali sono clausole di Horn. Un nodo è allagato se la sua zona ha pioggia forte o alluvione; un arco è transitabile da qualunque veicolo se nessuno dei due estremi è allagato, e solo dai mezzi pesanti altrimenti:

```
strada_allagata(Nodo) :- zona(Nodo, Zona), condizione_meteo(Zona, pioggia_forte).
strada_allagata(Nodo) :- zona(Nodo, Zona), condizione_meteo(Zona, alluvione).

transitabile(X, Y, V) :- arco_bidirezionale(X, Y, _), tipo_veicolo(V, _),
                         \+ strada_allagata(X), \+ strada_allagata(Y).
transitabile(X, Y, V) :- arco_bidirezionale(X, Y, _), tipo_veicolo(V, mezzo_pesante),
                         (strada_allagata(X) ; strada_allagata(Y)).

costo_arco(X, Y, V, C) :- arco_bidirezionale(X, Y, D), transitabile(X, Y, V),
                          calcola_maggiorazione(X, Y, V, D, C).
```

Il costo di un arco è la distanza moltiplicata per il fattore meteo peggiore tra i due estremi (nebbia 1,5; pioggia 1,3; pioggia forte 1,4; alluvione 1,6) e per 2 se un mezzo pesante attraversa una zona allagata. La regola `meteo_nodo/2` deduce il meteo di un nodo dalla sua zona ed è usata per costruire le feature del modello predittivo; `emergenza_critica/2` segnala all'operatore le emergenze gravi in zone con meteo avverso.

La negazione `\+` è **negazione come fallimento**: la KB adotta l'ipotesi di mondo chiuso, per cui un nodo non dimostrabilmente allagato è considerato praticabile. È un'assunzione ragionevole perché la centrale conosce il meteo di tutte le zone.

KB come unica fonte di verità. Nella prima versione del progetto nodi, zone, basi, capacità e tipi di emergenza erano ripetuti anche nel codice Python; ora vengono letti dalla KB (`get_tipi_emergenza`, `get_basi`, `get_meteo_nodo`, …), così una modifica alla KB si propaga a tutto il sistema.

Vincolo sui dati per l'euristica. Ogni arco deve avere distanza maggiore o uguale alla distanza euclidea tra le coordinate dei suoi estremi, altrimenti l'euristica di A* non è ammissibile (sezione 4). Nella prima versione 4 archi su 26 violavano la condizione (es. `ospedale–incrocio_nord`: distanza 3, distanza in linea d'aria 3,16); sono stati corretti e la condizione è verificata da un test.

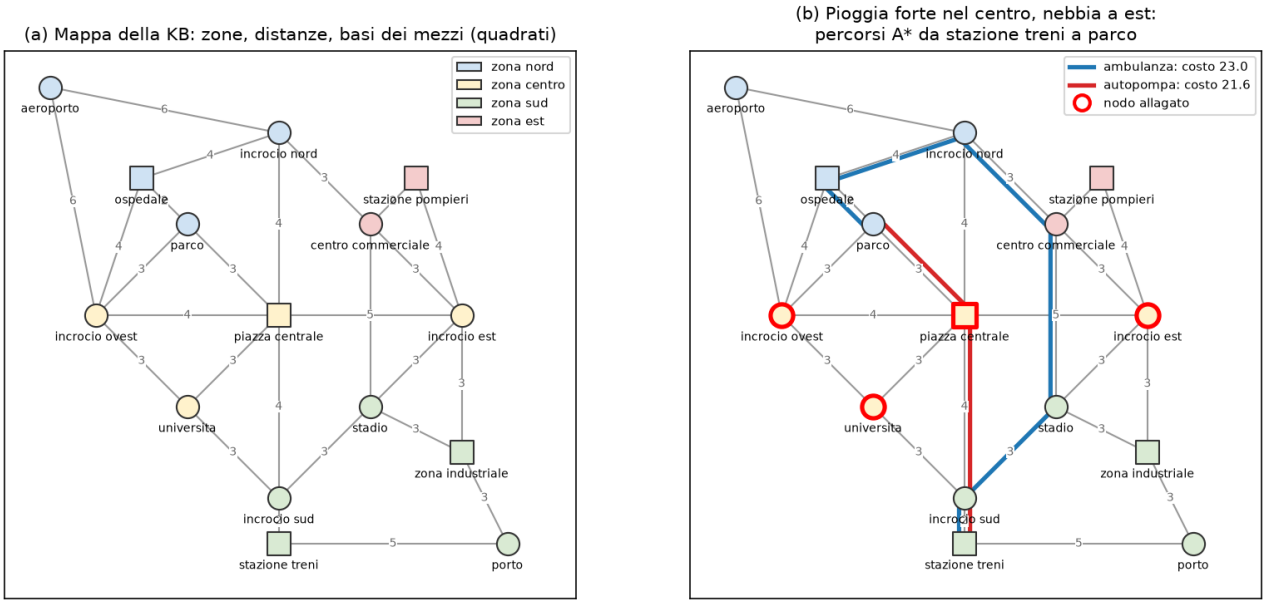


Figura 2 – (a) La mappa descritta nella KB. (b) Effetto delle regole con pioggia forte nel centro: i nodi allagati (bordo rosso) non sono transitabili per l'ambulanza, che aggira il centro (costo 23,0), mentre l'autopompa lo attraversa (costo 21,6) pur con il costo raddoppiato dall'allagamento.

Valutazione

La correttezza delle regole è verificata da 17 test (`test/test_kb.py`): deduzione dell'allagamento dopo un cambio di meteo, transitabilità per mezzi leggeri e pesanti, costo infinito per archi inesistenti, `assert`/`retract` del meteo e della fascia oraria, compatibilità delle distanze con l'euristica, alluvione più costosa della pioggia.

Ontologia (OWL)

Sommario

L'ontologia (`ontologia/ontologia_emergenze.py`, salvata in `ontologia/emergenze.owl` e apribile con Protégé) descrive in logica descrittiva il vocabolario del dominio: luoghi, emergenze e mezzi con le loro capacità operative.

Strumenti utilizzati

Owlready2 per costruire e interrogare l'ontologia in Python e il ragionatore HermiT (incluso in Owlready2, richiede Java).

Decisioni di progetto

- **TBox scritta a mano**: gerarchie `Mezzo` ⊒ `MezzoLeggero` ⊒ `Ambulanza`, `Mezzo` ⊒ `MezzoPesante` ⊒ {`Autopompa`, `Autoscala`}; `Emergenza` con 5 sottoclassi; `Luogo` ⊒ {`Nodo`, `Zona`}. Classi sorelle dichiarate disgiunte.
- **Proprietà** `situatoIn`, `avvieneIn`, `puoGestire`, `haBase` con dominio e codominio.
- **Restrizioni esistenziali** per le capacità operative, ad esempio `Autopompa` ⊑ ∃`puoGestire.Incendio`.
- **Classi definite** che il ragionatore popola da solo: `MezzoAntincendio` ≡ `Mezzo` ⊓ ∃`puoGestire.Incendio`, `MezzoSoccorsoSanitario`, `MezzoPolivalente` ≡ `Mezzo` ⊓ ∃`puoGestire.Incendio` ⊓ ∃`puoGestire.Incidente`.
- **ABox popolata dalla KB Prolog**: i 15 nodi e le 4 zone con la relazione `situatoIn`, senza riscriverli a mano.

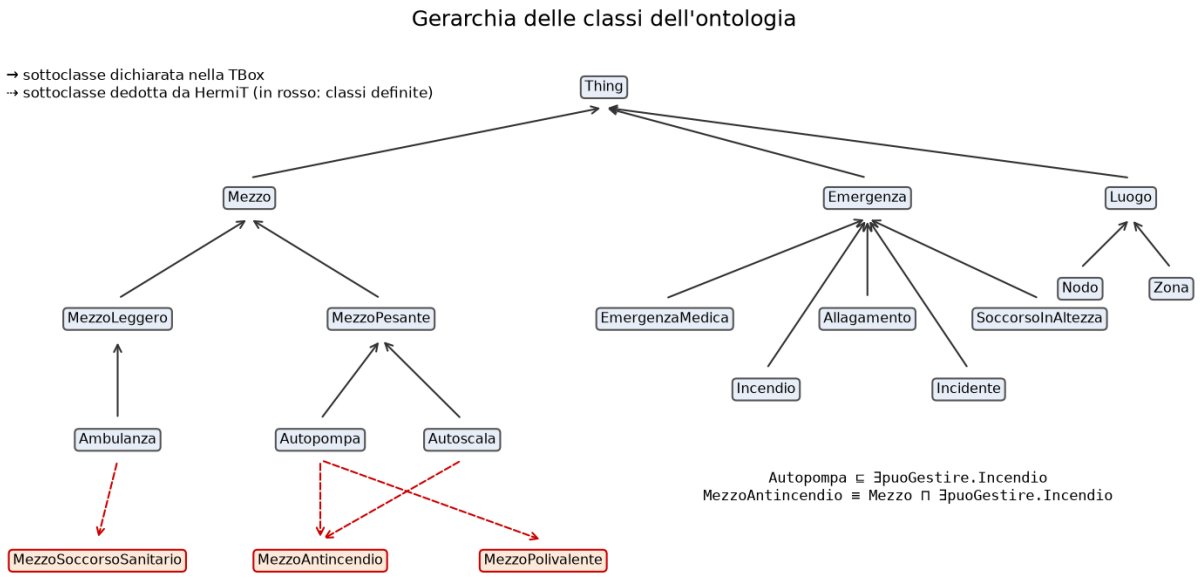


Figura 3 – Gerarchia delle classi: in nero le relazioni dichiarate, in rosso tratteggiato quelle dedotte da HermiT per le classi definite.

Ontologia e KB descrivono in parte la stessa conoscenza (quale mezzo gestisce quale emergenza) con ipotesi diverse: Prolog è a **mondo chiuso** (ciò che non è derivabile è falso), OWL a **mondo aperto** (ciò che non è affermato è sconosciuto). Per questo dall'ontologia si possono derivare le compatibilità ma non le incompatibilità. Il sistema confronta le due rappresentazioni su tutte le 15 coppie tipo di mezzo / tipo di emergenza.

Valutazione

Dopo la classificazione HermiT deduce che `Autopompa` e `Autoscala` sono sottoclassi di `MezzoAntincendio`, `Ambulanza` di `MezzoSoccorsoSanitario` e solo `Autopompa` di `MezzoPolivalente`, senza che queste relazioni siano scritte esplicitamente. Il confronto con `compatibile/2` della KB non rileva **nessuna discrepanza** su 15 coppie: le due basi di conoscenza sono coerenti. Il controllo viene eseguito a ogni avvio, quindi un errore introdotto in una delle due viene segnalato subito.

Ricerca del percorso (A*)

Sommario

Per ogni coppia mezzo/emergenza il sistema cerca il percorso di costo minimo nel grafo della città con A* (`ricerca/astar_routing.py`). Transitabilità e costo di ogni arco non sono letti da una tabella ma chiesti alla KB a ogni espansione, quindi il percorso tiene conto di meteo, allagamenti e tipo di veicolo.

Strumenti utilizzati

Implementazione propria in Python con coda di priorità (`heapq`) e interrogazioni Prolog tramite pyswip.

Decisioni di progetto

La frontiera è ordinata per f(n) = g(n) + h(n), dove h(n) è la distanza euclidea dal nodo alla destinazione. L'euristica è **ammissibile** perché ogni arco ha distanza ≥ della distanza euclidea tra i suoi estremi e tutti i fattori di costo sono ≥ 1, quindi h non sovrastima mai il costo reale. È anche **consistente** (monotona): per la disuguaglianza triangolare h(n) ≤ costo(n, n') + h(n'). Grazie alla consistenza un nodo estratto dalla frontiera ha già il costo minimo e non va mai riaperto, il che giustifica l'uso dell'insieme dei nodi chiusi.

Con il parametro `usa_euristica=False` la stessa procedura diventa la **ricerca a costo minimo** (h = 0), usata come riferimento: è ottimale ma non sfrutta alcuna informazione sulla destinazione.

Valutazione

Sono state confrontate le due ricerche su tutte le 210 coppie ordinate di nodi, per i 3 tipi di veicolo e 5 scenari meteo casuali (3150 ricerche per algoritmo, di cui 516 con destinazione irraggiungibile per un mezzo leggero a causa degli allagamenti).

Algoritmo	Nodi espansi	Tempo (ms)
A*	4.98 ± 3.34	10.76 ± 7.24
Costo minimo	7.42 ± 4.36	11.81 ± 7.64

Tabella 1 – A contro ricerca a costo minimo (media ± deviazione standard).*

- In **0 casi** A* ha restituito un costo diverso dalla ricerca a costo minimo: le soluzioni sono sempre ottime, come garantito dall'ammissibilità.
- In **0 casi** h(n) ha superato il costo reale del percorso: verifica empirica dell'ammissibilità dopo la correzione dei dati (prima della correzione le coppie che la violavano erano 10).

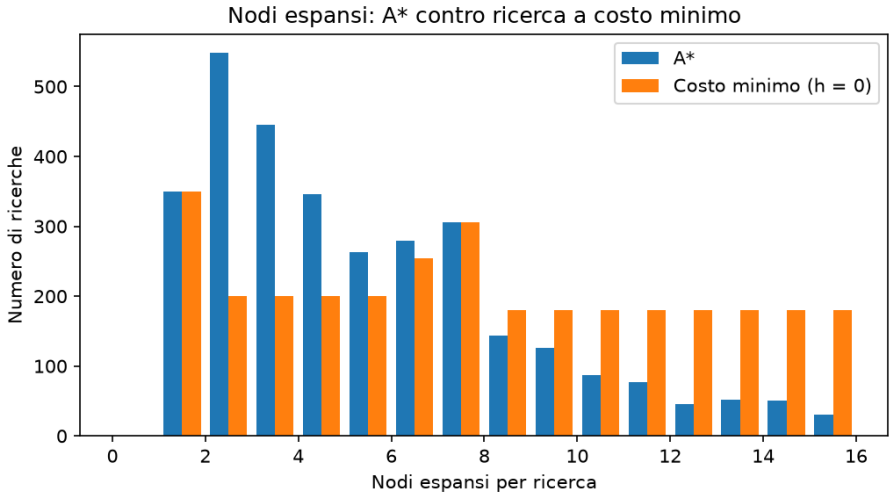


Figura 4 – Distribuzione dei nodi espansi per ricerca: l'euristica sposta la distribuzione verso valori più bassi.

- L'euristica riduce i nodi espansi del **32,8%**. Il guadagno di tempo è minore, perché ogni calcolo di h(n) è a sua volta una query Prolog: su un grafo di 15 nodi il costo delle query domina quello della ricerca.

Allocazione dei mezzi (CSP)

Sommario

Assegnare i mezzi alle emergenze è un problema combinatorio: con m mezzi ed n emergenze ci sono (m+1)^n assegnamenti possibili. È modellato come problema di soddisfacimento di vincoli con funzione obiettivo e risolto in due modi: in modo esatto, con propagazione dei vincoli e branch and bound (`ricerca/csp_backtracking.py`), e con ricerca locale (`ricerca/csp_allocazione.py`).

Strumenti utilizzati

Implementazione propria in Python di AC-3, backtracking con forward checking, branch and bound, Random Restart Hill Climbing e Simulated Annealing.

Decisioni di progetto

- **Variabili**: le emergenze. **Domini**: i mezzi più il valore `non_assegnato`.

- **Vincoli hard**: compatibilità mezzo/emergenza (dalla KB), raggiungibilità (costo A* finito), capacità del mezzo (1 per ambulanza e autoscala, 2 per l'autopompa), copertura obbligatoria delle emergenze di gravità alta.
- **Obiettivo da minimizzare**: somma dei costi A* pesati per gravità (alta 3, media 2, bassa 1), penalità soft oltre la distanza massima di 25, penalità di 50 × peso per ogni emergenza non assegnata.

Risoluzione esatta. I vincoli unari (compatibilità, raggiungibilità, gravità alta) sono applicati subito ai domini (consistenza di nodo). Sui vincoli binari di capacità (due emergenze non possono usare lo stesso mezzo di capacità 1) si applica **AC-3**: se un dominio si svuota il problema è insoddisfacibile e lo si scopre senza alcuna ricerca. Segue un **backtracking** che sceglie per prima la variabile con il dominio più piccolo (euristica MRV) e applica il **forward checking**: quando un mezzo esaurisce la capacità viene tolto dai domini delle variabili non ancora assegnate. Il **branch and bound** pota i rami il cui costo parziale più una stima ottimistica del costo restante (il minimo di ogni dominio) è già peggiore della migliore soluzione trovata; poiché la stima non sovrastima mai, la potatura non perde l'ottimo.

Ricerca locale. RRHC e SA esplorano gli assegnamenti completi cambiando una variabile alla volta; i vincoli hard sono trasformati in penalità. Una prova sperimentale ha rivelato un errore di modellazione della prima versione: la penalità per violazione (100) era uguale a quella per lasciare non assegnata un'emergenza di gravità media (50 × 2), quindi la ricerca locale trovava conveniente violare un vincolo, ad esempio inviando un'ambulanza verso un nodo irraggiungibile. La penalità è stata portata a 1000, in modo che domini qualunque costo ammissibile.

Valutazione

Su 30 scenari casuali con 7 emergenze, 7 mezzi e meteo casuale, l'ottimo del branch and bound è stato usato come riferimento per misurare la qualità della ricerca locale. In 3 scenari su 30 non esiste alcuna allocazione ammissibile (le emergenze gravi sono più dei mezzi compatibili): AC-3 o il backtracking lo dimostrano e quegli scenari sono esclusi dalla tabella. In simulazione, in questo caso, il sistema rilassa solo il vincolo sulle emergenze gravi, calcola la migliore allocazione possibile e segnala le emergenze gravi rimaste scoperte come da affidare a rinforzi esterni.

Algoritmo	Costo	Scarto dall'ottimo (%)	Ottimo trovato	Soluzione ammissibile	Tempo (ms)
Branch and bound	165.15 ± 69.75	0.00 ± 0.00	27/27	27/27	1.59 ± 1.57
Random Restart HC	165.23 ± 69.66	0.10 ± 0.49	26/27	27/27	125.51 ± 13.26
Simulated Annealing	171.52 ± 78.29	2.84 ± 5.67	20/27	27/27	69.94 ± 4.93

Tabella 2 – Allocazione: branch and bound contro ricerca locale sugli scenari risolvibili.

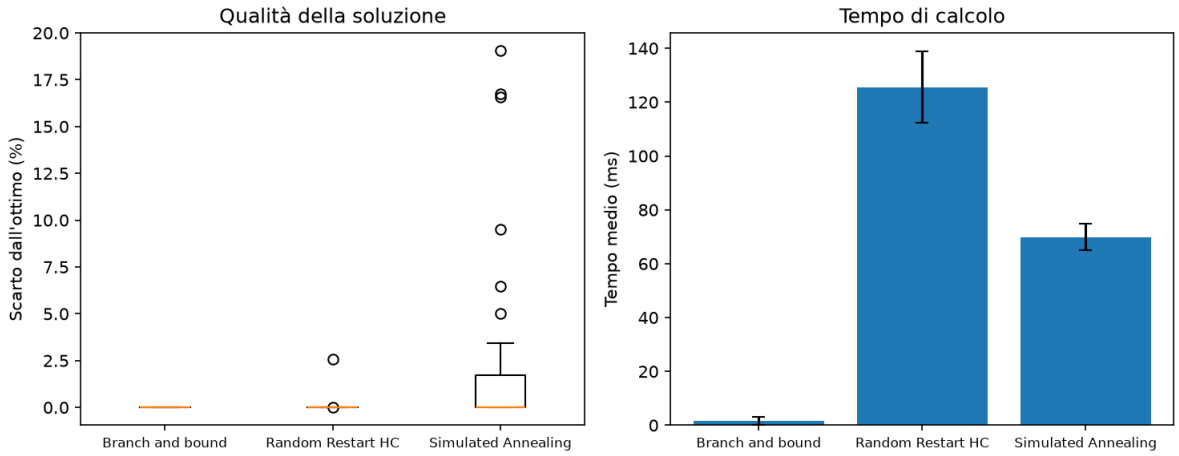


Figura 5 – Scarto dall'ottimo e tempo di calcolo dei tre algoritmi sugli scenari risolvibili.

- Il branch and bound trova l'ottimo visitando in media 46 nodi su milioni di assegnamenti possibili, grazie a propagazione e potatura, ed è anche il più veloce.

- RRHC raggiunge quasi sempre l'ottimo grazie ai 30 riavvii; SA, con un unico percorso e un raffreddamento geometrico, resta più spesso in un ottimo locale.
- A questa scala la ricerca locale non conviene; il suo vantaggio emerge su istanze molto grandi, dove la ricerca esatta cresce esponenzialmente mentre il costo della ricerca locale resta controllabile con il numero di iterazioni.

Apprendimento supervisionato

Sommario

Obiettivo: stimare per ogni emergenza la probabilità che serva **supporto aggiuntivo** (un secondo mezzo o rinforzi), così da allertare in anticipo le risorse. Feature: meteo, fascia oraria, zona, gravità, tipo di emergenza, ritardo di arrivo; target binario.

Strumenti utilizzati

scikit-learn (pipeline, GridSearchCV, StratifiedKFold, classificatori), pandas, matplotlib.

Decisioni di progetto

Dataset. Non essendo disponibile uno storico reale di interventi, il dataset (2000 esempi) è generato da un modello causale esplicito, dichiarato nel codice (`dataset_synth.py`): meteo, zona e orario determinano in modo probabilistico il ritardo di arrivo; gravità, tipo, ritardo e orario determinano la probabilità di supporto, da cui il target è estratto a caso. Le prestazioni misurate valgono quindi per questo modello generativo, non per dati reali; il vantaggio è conoscere il **limite teorico**: il classificatore bayesiano ottimo, che conosce le vere probabilità, ottiene su questi dati accuracy 0,689 e AUC 0,742. Nessun modello può fare meglio, perché il target contiene rumore irriducibile.

Collegamento con il resto del sistema. In simulazione la feature `ritardo_arrivo` è ottenuta discretizzando il costo A* del mezzo assegnato dal CSP (basso < 8, medio < 15, alto altrimenti), e meteo, zona e fascia oraria sono letti dalla KB.

Data cleaning. Il generatore lascia mancante circa il 3% dei valori di meteo, orario e zona, come accade con schede compilate in fretta. I valori mancanti sono sostituiti con la moda **dentro la pipeline**: la moda è così calcolata solo sui fold di training e non c'è fuga di informazione verso i fold di test. Le righe duplicate non vengono rimosse perché in un registro di eventi due righe uguali sono interventi distinti.

Valori fuori dominio. Se in predizione arriva un valore non previsto (ad esempio un meteo `sconosciuta` per un nodo assente dalla KB), il primo passo della pipeline lo trasforma in valore mancante, che viene poi imputato: Naive Bayes e rete bayesiana, che altrimenti fallirebbero su una categoria mai vista, restano utilizzabili.

Codifica. One-hot per alberi, Random Forest e regressione logistica; codifica ordinale per il Naive Bayes categorico, che stima una distribuzione per ogni valore di ciascuna feature.

Modelli (iperparametri scelti nel ciclo interno): albero di decisione (profondità massima, minimo di esempi per foglia, criterio di impurità), Random Forest (profondità, minimo per foglia), regressione logistica (forza della regolarizzazione C), Naive Bayes categorico (smoothing di Laplace α). Come riferimento una baseline che predice sempre la probabilità a priori della classe.

Valutazione con nested cross-validation. Ciclo esterno StratifiedKFold a 5 fold per stimare le prestazioni, ciclo interno GridSearchCV a 3 fold per scegliere gli iperparametri usando solo il training del fold esterno. Tutti i modelli usano gli stessi fold. Per ogni metrica si riportano media e deviazione standard sui 5 fold esterni.

Scelta della metrica. In una prima versione gli iperparametri erano scelti massimizzando F1. La baseline ha mostrato che non è adatto: con classi bilanciate un classificatore che risponde sempre "supporto" ha recall

1 e F1 0,668, più di tutti i modelli. Si usa quindi l'**AUC-ROC**, che misura quanto bene il modello ordina i casi per rischio indipendentemente dalla soglia: per una centrale operativa conta proprio sapere quali emergenze sono più a rischio.

Valutazione

Modello	accuracy	precision	recall	f1	log_loss	roc_auc
Albero di decisione	0.665 ± 0.021	0.733 ± 0.035	0.524 ± 0.018	0.611 ± 0.023	0.663 ± 0.060	0.721 ± 0.025
Random Forest	0.662 ± 0.019	0.681 ± 0.020	0.612 ± 0.023	0.645 ± 0.021	0.614 ± 0.010	0.730 ± 0.021
Regressione logistica	0.666 ± 0.024	0.678 ± 0.024	0.637 ± 0.030	0.657 ± 0.027	0.624 ± 0.006	0.735 ± 0.017
Naive Bayes	0.670 ± 0.022	0.689 ± 0.021	0.622 ± 0.029	0.654 ± 0.025	0.604 ± 0.014	0.734 ± 0.017
Rete bayesiana (esperta)	0.619 ± 0.035	0.621 ± 0.036	0.617 ± 0.047	0.619 ± 0.040	0.643 ± 0.017	0.674 ± 0.034
Rete bayesiana (appresa, BIC)	0.652 ± 0.028	0.694 ± 0.059	0.566 ± 0.042	0.620 ± 0.019	0.613 ± 0.020	0.725 ± 0.025
Baseline (probabilità a priori)	0.501 ± 0.001	0.501 ± 0.001	1.000 ± 0.000	0.668 ± 0.001	0.693 ± 0.000	0.500 ± 0.000

Tabella 3 – Nested CV 5×3: media ± deviazione standard sui fold esterni.

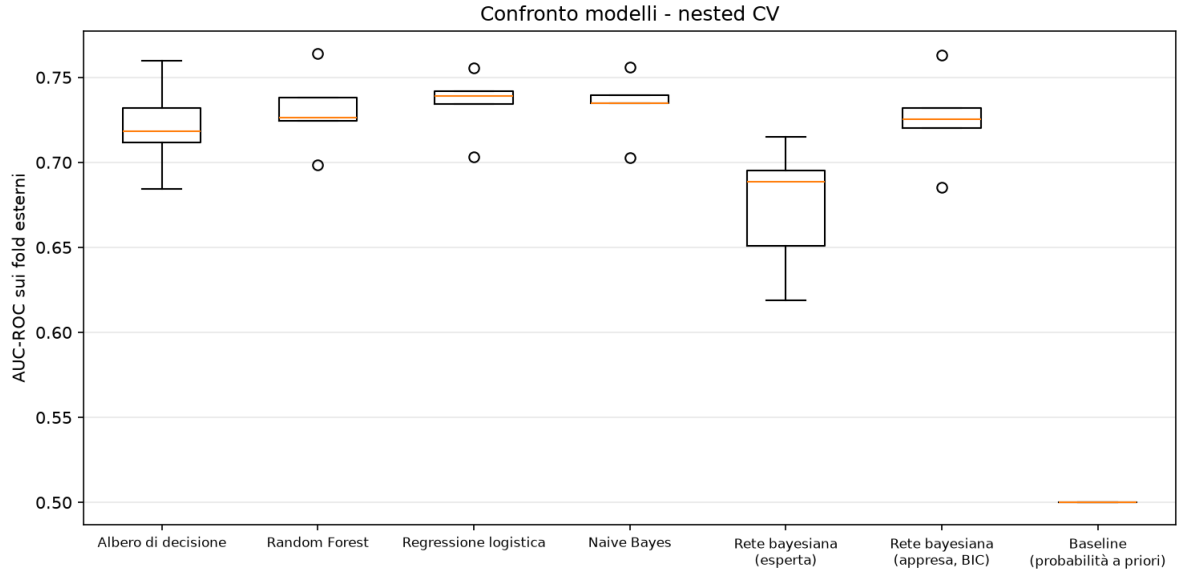


Figura 6 – Distribuzione dell'AUC-ROC sui fold esterni per ogni modello.

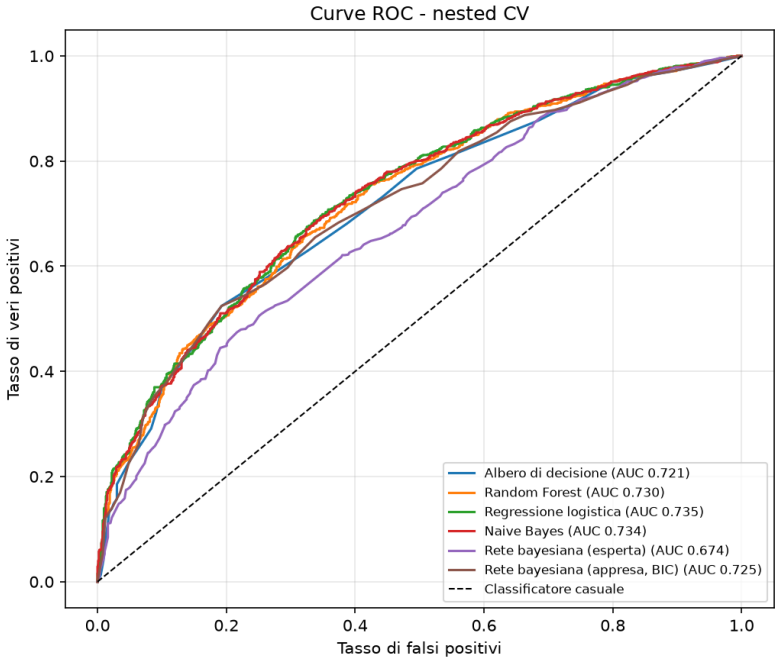


Figura 7 – Curve ROC calcolate sulle predizioni dei fold esterni.

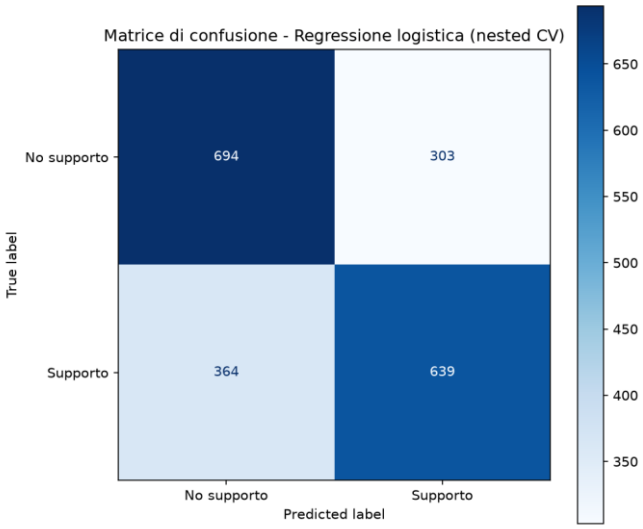


Figura 8 – Matrice di confusione del modello scelto (regressione logistica) sui fold esterni, soglia 0,5.

- I modelli migliori raggiungono un'AUC di circa 0,735, a ridosso del limite teorico di 0,742: su questi dati non c'è margine per modelli più complessi.
- Regressione logistica e Naive Bayes eguagliano o superano Random Forest e albero: il modello generativo è additivo (ogni fattore aggiunge una quota di probabilità), quindi i modelli con un bias induttivo additivo sono favoriti e la maggiore flessibilità degli alberi si traduce solo in varianza.
- L'albero di decisione ha la log loss peggiore tra i classificatori: le sue foglie stimano probabilità da pochi esempi, quindi poco calibrate.
- Le deviazioni standard dei classificatori sono intorno a 0,02: le differenze tra i modelli migliori sono dello stesso ordine e non vanno sopravvalutate.

Ragionamento con incertezza (rete bayesiana)

Sommario

Lo stesso problema è affrontato con una rete bayesiana (`machine_learning/rete_bayesiana.py`), che rappresenta la distribuzione congiunta delle variabili con un grafo orientato aciclico e una tabella di probabilità condizionata (CPT) per nodo. A differenza dei classificatori la rete rende esplicite le dipendenze e risponde a query con **evidenza parziale**.

Strumenti utilizzati

pgmpy (stima dei parametri, ricerca della struttura, eliminazione delle variabili), integrata in scikit-learn con una classe compatibile (`fit`/`predict_proba`) per valutarla con la stessa nested CV.

Decisioni di progetto

- **Struttura esperta**: definita dalla conoscenza del dominio, meteo, zona, orario → ritardo_arrivo; gravità, tipo, ritardo, orario → supporto.
- **Struttura appresa**: Hill Climbing sullo spazio dei grafi che massimizza il punteggio **BIC** (log-verosimiglianza meno (log N / 2) × numero di parametri), che bilancia adattamento ai dati e complessità.
- **Parametri**: stimatore bayesiano con prior BDeu; la forza del prior (equivalent sample size) è l'iperparametro scelto nel ciclo interno. I pseudo-conteggi evitano probabilità nulle per combinazioni mai osservate.
- **Inferenza**: eliminazione delle variabili. Per una query si sommano le variabili non osservate; ad esempio P(supporto | meteo = alluvione) si calcola senza conoscere orario, zona o gravità.

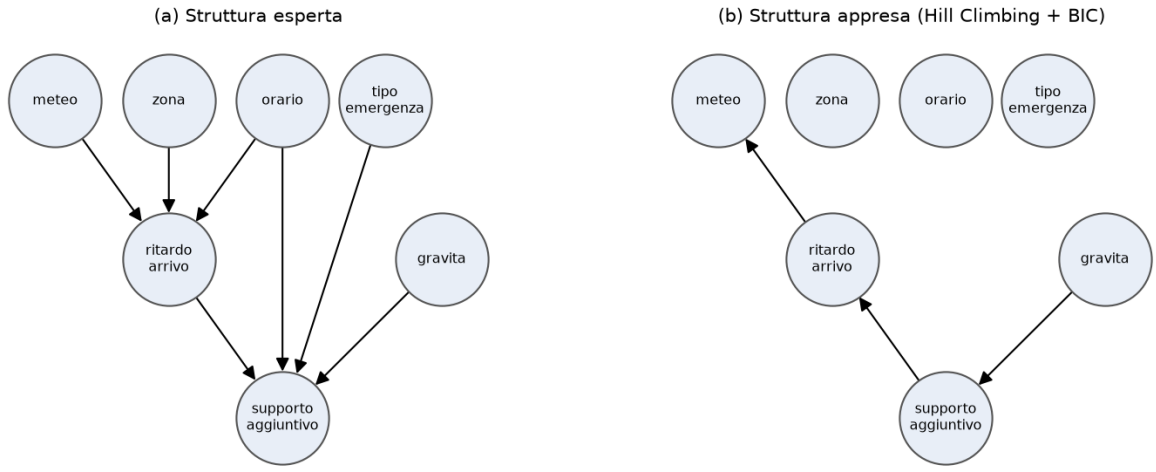


Figura 9 – Struttura esperta (a sinistra) e struttura appresa sull'intero dataset con Hill Climbing e BIC (a destra).

La struttura appresa collega al supporto solo gravità e ritardo, e il meteo attraverso il ritardo; zona, orario e tipo di emergenza restano isolati. Nel generatore anche queste variabili influiscono, ma con effetti piccoli: con 2000 esempi il BIC giudica che il miglioramento della verosimiglianza non valga i parametri in più. Gli archi appresi sono inoltre orientati in senso opposto alla causalità (supporto → ritardo → meteo): reti nella stessa classe di equivalenza di Markov hanno lo stesso punteggio BIC, quindi dai soli dati osservazionali la direzione degli archi non è identificabile e non va letta in senso causale.

Valutazione

Nella Tabella 3 la rete con struttura esperta ottiene un'AUC più bassa dei classificatori, nonostante la sua struttura sia proprio quella usata per generare i dati. Il motivo è il numero di parametri: il nodo supporto ha 4 genitori con 3 × 5 × 3 × 4 = 180 configurazioni, cioè circa 11 esempi per configurazione con 2000 esempi.

Le stime sono rumorose e il ciclo interno sceglie infatti il prior più forte tra quelli provati. La struttura appresa con BIC è più povera ma generalizza meglio. È il compromesso tra bias e varianza: con pochi dati un modello sbagliato ma semplice batte un modello corretto ma complesso. La curva di apprendimento lo conferma:

Esempi	Rete bayesiana (esperta)	Rete bayesiana (appresa, BIC)	Regressione logistica
500	0.665 ± 0.024	0.708 ± 0.012	0.737 ± 0.023
2000	0.701 ± 0.005	0.735 ± 0.019	0.743 ± 0.018
8000	0.745 ± 0.002	0.757 ± 0.003	0.758 ± 0.001
20000	0.756 ± 0.006	0.761 ± 0.006	0.762 ± 0.007

Tabella 4 – AUC-ROC (nested CV 3×2) al crescere del numero di esempi.

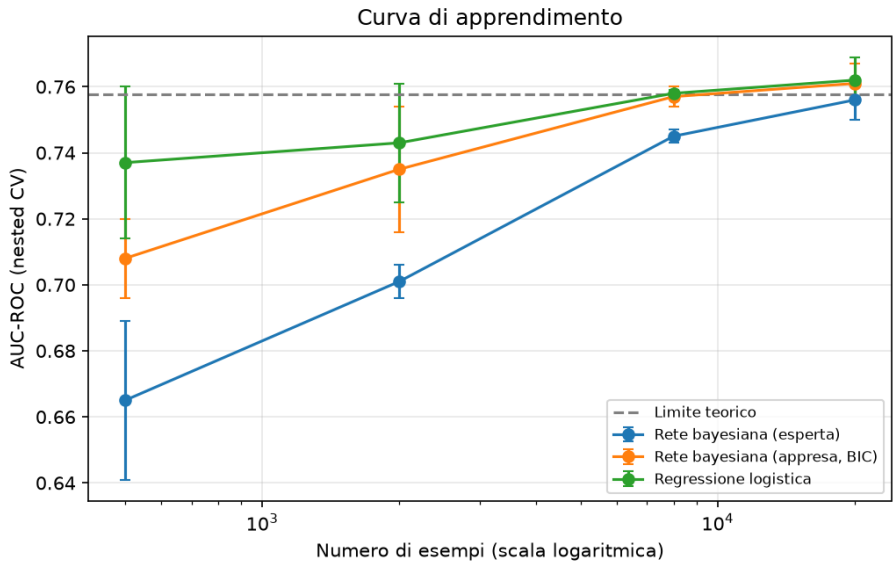


Figura 10 – Curva di apprendimento (asse orizzontale in scala logaritmica); la linea tratteggiata è il limite teorico.

Con 500 esempi la rete esperta è nettamente la peggiore (AUC 0,665); al crescere dei dati le sue CPT diventano affidabili e il divario si chiude: con 20000 esempi tutti e tre i modelli arrivano a circa 0,76, cioè al limite teorico del generatore (0,759, stimato su 200000 esempi); gli scarti di qualche millesimo in eccesso rientrano nella deviazione standard. La regressione logistica, con pochi parametri, è già vicina al massimo con 2000 esempi.

Esempi di query con evidenza parziale sulla rete esperta addestrata sull'intero dataset:

Evidenza	P(supporto | evidenza)
nessuna evidenza (probabilità a priori)	0.500
meteo = alluvione	0.515
gravita = alta	0.661
gravita = alta, meteo = alluvione, orario = notte	0.680
gravita = bassa, ritardo_arrivo = basso	0.287

Tabella 5 – Inferenza per eliminazione delle variabili.

Conclusioni

Il sistema integra rappresentazione della conoscenza (Prolog e OWL), ricerca (A*), vincoli (AC-3, backtracking, branch and bound, ricerca locale) e apprendimento (classificatori e reti bayesiane) in un'unica simulazione in cui ogni modulo usa i risultati degli altri. La valutazione sperimentale ha confermato le proprietà teoriche attese (ottimalità di A* con euristica ammissibile, ottimalità del branch and bound) e ha fatto emergere diversi errori della prima versione, poi corretti: un'euristica non ammissibile per colpa dei dati, un generatore di scenari che produceva una sola ambulanza su sei mezzi, una penalità dei vincoli

troppo bassa nella ricerca locale, una metrica di selezione (F1) premiata anche da un classificatore banale e modelli che fallivano su valori mai visti. Anche il caso in cui le risorse non bastano a coprire le emergenze gravi è ora gestito esplicitamente.

Sviluppi futuri:

- usare dati reali (open data su interventi di soccorso o incidenti stradali) al posto del dataset sintetico;
- estrarre la mappa da OpenStreetMap e generare automaticamente i fatti Prolog;
- ricalcolare percorsi e allocazione quando il meteo cambia durante l'intervento;
- con dati reali in cui manchi il ritardo osservato, trattarlo come variabile latente e stimarne le CPT con l'algoritmo EM.

Riferimenti bibliografici

[1] D. Poole, A. Mackworth. Artificial Intelligence: Foundations of Computational Agents, 3rd ed. Cambridge University Press, 2023. https://artint.info/3e/html/ArtInt3e.html

[2] D. Poole, A. Mackworth. AIPython: Python code for AI foundations of computational agents. https://artint.info/AIPython/

[3] SWI-Prolog e pyswip. https://www.swi-prolog.org, https://github.com/yuce/pyswip

[4] J.-B. Lamy. Owlready: Ontology-oriented programming in Python with automatic classification and high level constructs for biomedical ontologies. Artificial Intelligence in Medicine 80, 2017.

[5] F. Pedregosa et al. Scikit-learn: Machine Learning in Python. JMLR 12, 2011. https://scikit-learn.org

[6] A. Ankan, J. Textor. pgmpy: A Python Toolkit for Bayesian Networks. JMLR 25, 2024. https://pgmpy.org